

HAI811I – Programmation mobile

Rapport final de projet

Robin Dejean et Léo Hafdane

[*https://github.com/Leottaro/HAI811I_Projet*](https://github.com/Leottaro/HAI811I_Projet)

16 mai 2026

M1 Informatique – IMAGINE
Faculté des Sciences
Université de Montpellier



Résumé du projet

Ce projet de programmation mobile repose sur la réalisation de deux applications Android complémentaires : **TravelShare** et **TravelPath**. Les applications ont été développées en Kotlin avec Jetpack Compose et s'appuient sur une architecture moderne orientée composants et services.

Le travail réalisé couvre l'ensemble du cycle de développement :

- conception de l'architecture applicative ;
- mise en place de l'authentification et de la gestion des utilisateurs ;
- intégration d'une base de données temps réel avec Firebase Firestore ;
- gestion du stockage et de l'upload des médias via Cloudinary ;
- développement d'interfaces modernes avec Jetpack Compose ;
- implémentation de fonctionnalités sociales et de navigation ;
- optimisation de la gestion des états et du cache local.

Les deux projets partagent plusieurs composants techniques communs afin de garantir une cohérence de développement et de faciliter la maintenance du code.

Fonctionnalités implémentées

Fonctionnalités communes

- Authentification utilisateur avec Firebase Authentication ;
- Création et personnalisation de profils utilisateurs ;
- Gestion des sessions et déconnexion sécurisée ;
- Upload et stockage d'images via Cloudinary ;
- Synchronisation des données avec Firebase Firestore ;
- Interfaces modernes développées avec Jetpack Compose ;
- Navigation dynamique entre les écrans ;
- Gestion des permissions Android ;
- Notifications push avec Firebase Cloud Messaging ;
- Gestion des erreurs et des états de chargement.

Fonctionnalités spécifiques à TravelShare

- Publication et partage de contenus multimédias ;
- Consultation des publications des autres utilisateurs ;
- Système d'interactions sociales ;
- Gestion des profils et des contenus associés ;
- Rafraîchissement dynamique des données utilisateur.

Fonctionnalités spécifiques à TravelPath

- Gestion d'itinéraires et de favoris ;
- Mise en cache locale avec Room ;
- Synchronisation des données utilisateur ;
- Sauvegarde et restauration des préférences ;
- Gestion optimisée des données hors ligne.

Architecture et choix techniques

Architecture générale

Les applications suivent une architecture modulaire inspirée du modèle MVVM (Model – View – ViewModel). Cette organisation permet de séparer clairement :

- la logique métier ;
- l'interface utilisateur ;
- l'accès aux données ;
- les services externes.

Jetpack Compose est utilisé pour la couche interface utilisateur tandis que Firebase et Cloudinary assurent la gestion des données distantes et des médias.

Technologies utilisées

- **Kotlin** : langage principal des applications ;
- **Jetpack Compose** : développement déclaratif des interfaces ;
- **Firebase Authentication** : gestion des comptes utilisateurs ;
- **Firebase Firestore** : stockage des données ;
- **Firebase Cloud Messaging** : notifications push ;
- **Cloudinary** : hébergement et optimisation des médias ;
- **Room** : persistance locale des données ;
- **Coil** : chargement et affichage des images.

Choix de conception

Plusieurs choix techniques ont été réalisés afin d'assurer la maintenabilité et les performances des applications :

- centralisation des services externes ;
- séparation des responsabilités entre composants ;
- gestion réactive des états avec Compose ;
- utilisation d'un cache local pour limiter les accès réseau ;
- sécurisation des échanges via HTTPS ;
- externalisation des clés sensibles dans `local.properties`.

Travail réalisé

Général

Les deux sous-projets partagent une infrastructure commune reposant sur Firebase (Auth, Firestore, Cloud Messaging) et Cloudinary pour le stockage des médias.

Authentification utilisateur

- **Authentification Firebase.** Les deux applications s'appuient sur le SDK `firebase-auth` (Kotlin). L'état de session est observé dans `MainActivity` via un `mutableStateOf` Compose : à la connexion, l'interface principale est chargée ; à la déconnexion, l'`AuthScreen` reprend la main.
- **Gestion de session.** `TravelShare` utilise `key(currentUser.uid)` pour forcer la recréation complète du graphe de navigation lors d'un changement d'utilisateur, évitant les fuites de `ViewModel`. `TravelPath` purge le cache local `Room` des favoris au logout pour éviter toute fuite de données entre comptes.
- **Profils personnalisables.** Les deux applications maintiennent un document utilisateur dans Firestore (`/users/{uid}`) avec pseudo, bio et photo de profil. La photo est uploadée sur Cloudinary avant que l'URL sécurisée soit persistée dans Firestore.
Dépendances communes : `com.google.firebase:firebase-auth-ktx`, `com.google.firebase:firebase-firestore-ktx`, `com.cloudinary:cloudinary-android`, `io.coil-kt:coil-compose`.

Upload de médias (Cloudinary)

- **Initialisation.** `MediaManager.init()` est appelé dans `TravelWowApplication.onCreate()` avec le `cloud_name` issu de `BuildConfig` (injecté depuis `local.properties` via Gradle, non versionné). L'option `secure = true` force l'utilisation d'URLs HTTPS.
- **Upload.** Les deux projets envoient les images via `MediaManager` et stockent les URLs publiques retournées dans Firestore. `TravelPath` encapsule l'upload dans une coroutine suspendue (`suspendCancellableCoroutine`) ; `TravelShare` utilise `MediaManager.get().upload(uri).dispatch()`.
Dépendances : `com.cloudinary:cloudinary-android`.

Notifications

- **Firebase Cloud Messaging (FCM).** Les deux applications déclarent un service étendant `FirebaseMessagingService`. `onNewToken()` met à jour le champ `fcmToken` du document utilisateur Firestore. `onMessageReceived()` affiche une notification système locale.
Dépendances : `com.google.firebase:firebase-messaging-ktx`, `androidx.core:core-ktx`.
- **Notifications in-app temps réel.** Un écouteur Firestore (`addSnapshotListener` ou `callbackFlow`) surveille la collection `notifications` filtrée sur l'utilisateur courant et les entrées non lues. À chaque nouvelle notification, une notification système Android est déclenchée, puis le document est marqué comme lu. Le canal de notification (`NotificationChannel`) est créé pour Android 8+.
- **Permission Android 13+.** La permission `POST_NOTIFICATIONS` (requis depuis API 33) est demandée au démarrage via `ActivityResultContracts.RequestPermission()` dans `MainActivity`.
Dépendances : `androidx.activity:activity-ktx`, `androidx.core:core-ktx`.

Spécifique TravelShare

Authentification

- **Connexion email/mot de passe et inscription.** `FirebaseAuth.signInWithEmailAndPassword()` pour la connexion ; `createUserWithEmailAndPassword()` pour l'inscription, suivie de la création d'un document `UserProfile` dans Firestore. Le pseudo est vérifié en amont avec une requête `whereEqualTo("username", ...)` pour garantir l'unicité.
- **Mode anonyme.** `FirebaseAuth.signInAnonymously()` permet un accès limité. Le flag `isAnonymous` conditionne l'accès aux fonctionnalités connectées dans toute l'application.

Feed & Filtrage

- **Flux temps réel.** `PhotoRepository.getPhotosFlow()` utilise `addSnapshotListener` Firestore encapsulé dans un `callbackFlow` Kotlin. Le `ViewModel` collecte ce `Flow` et expose les photos via un `StateFlow<List<TravelPhoto>>`.
- **Filtrage par type de lieu.** Le champ `placeType: String` du modèle `TravelPhoto` (valeurs : "Nature", "Musée", "Rue", "Autre") permet un filtre côté client dans le `ViewModel`.
- **Recherche textuelle.** `PhotoRepository.searchPhotos()` récupère toutes les photos publiques, puis filtre en mémoire sur `description`, `locationName` et `tags` via `String.contains(ignoreCase = true)`.
Dépendances : `com.google.firebase:firebase-firestore`, `io.coil-kt:coil-compose`.

Vue carte (OSMDroid)

- **Carte interactive.** `MapView OSMDroid` embarquée dans un `AndroidView` Compose. Les tuiles sont fournies par `OpenStreetMap` (aucune clé API requise). La configuration `OSMDroid` est initialisée dans `TravelWowApplication` avec les `SharedPreferences` par défaut.

- **Markers.** Pour chaque `TravelPhoto` disposant de coordonnées non nulles, un `Marker` est ajouté. Un clic navigue vers `PhotoDetailScreen` avec le flag `fromMap = true` pour gérer correctement le retour arrière.

Dépendances : `org.osmdroid:osmdroid-android`, `org.osmdroid:osmdroid-wms`.

Détail d'une publication

- **Like / Unlike.** `PhotoRepository.toggleLike()` utilise une **transaction Firestore** (`db.runTransaction`) pour lire `likedBy`, incrémenter/décroître `likesCount` atomiquement et mettre à jour `FieldValue.arrayUnion/arrayRemove`, évitant les race conditions.
- **Signalement.** `PhotoRepository.reportPhoto()` crée un document dans la collection `reports` avec `photoId`, `reporterId`, `timestamp` et `status = "En attente"`.
- **Suppression / Modification.** Suppression via `photosCollection.document(photoId).delete()`. La modification navigue vers `UploadScreen` avec le `TravelPhoto` passé en paramètre.
- **Itinéraire.** Un `Intent(Intent.ACTION_VIEW)` avec l'URI `geo:0,0?q=<locationName>` ouvre Google Maps ou toute application de navigation installée.
- **Commentaires.** Flux temps réel depuis la sous-collection `photos/{photoId}/comments` via `PhotoRepository.getCommentsFlow()` (`pattern callbackFlow + addSnapshotListener`). L'ajout d'un commentaire appelle `addComment()` qui écrit dans la sous-collection puis déclenche une notification pour l'auteur.

Dépendances : `com.google.firebase:firebase-firestore`, `android.content.Intent`.

Publication

- **Sélection d'images.** `ActivityResultContracts.GetMultipleContents("image/*")` permet la sélection multi-images depuis la galerie.
- **Sauvegarde Firestore.** Un objet `TravelPhoto` est créé avec `authorId`, `authorName`, `imageUrls`, `locationName`, `placeType`, `tags`, `description`, `timestamp` et `isPublic = true`.
- **Mode édition.** `UploadScreen` accepte un paramètre optionnel `editingPhoto: TravelPhoto?`. Si présent, les champs sont pré-remplis et la sauvegarde effectue un `update()` sur le document existant au lieu d'un `add()`.

Dépendances : `com.cloudinary:cloudinary-android`, `com.google.firebase:firebase-firestore`

Social (amis)

- **Recherche d'utilisateurs.** `UserRepository.searchUsers(query)` utilise les opérateurs Firestore `whereGreaterThanOrEqualTo + whereLessThanOrEqualTo` avec le suffixe `"\uf8ff"` pour simuler une recherche par préfixe sur `username`. L'utilisateur courant est exclu des résultats.
- **Envoi de demande d'ami.** `FriendRepository.sendFriendRequest()` vérifie l'absence de doublon, crée un `FriendRequest` avec `status = "PENDING"` dans la collection `friendRequests`, puis crée une `AppNotification` pour le destinataire.

- **Acceptation / Refus.** L'acceptation utilise un **batch Firestore** atomique : passage du statut à "ACCEPTED" et ajout mutuel des UIDs dans les tableaux `friends` via `FieldValue.arrayUnion`. Le refus est un simple `delete()` du document `FriendRequest`.
- **Demandes entrantes.** `getIncomingRequestsFlow(userId)` écoute en temps réel les `friendRequests` où `toId == userId` et `status == "PENDING"`.
Dépendances : `com.google.firebase:firebase-firestore`.

Messagerie

- **Chat 1-to-1.** L'ID de conversation est déterminé de façon déterministe par `getChatId()` (tri lexicographique des UIDs : "uid1_uid2"), garantissant l'unicité. Les messages sont stockés dans `chats/{chatId}/messages`.
- **Chat de groupe.** Les groupes sont stockés dans la collection `groups`. `createGroup()` crée le document avec la liste des membres. Les messages sont dans `groups/{groupId}/messages`.
- **Flux temps réel.** `getMessagesFlow()` et `getGroupMessagesFlow()` utilisent `addSnapshotListener` avec tri par `timestamp ASC` encapsulés en `callbackFlow`.
- **Partage de photo.** `ChatMessage` dispose d'un champ optionnel `photoId: String?`. Le destinataire peut appuyer sur le message pour naviguer vers `PhotoDetailScreen`.
- **Notifications à l'envoi.** `sendMessage()` crée systématiquement une `AppNotification` après l'envoi. Pour les groupes, `sendGroupMessage()` crée une notification pour chaque membre sauf l'expéditeur.
Dépendances : `com.google.firebase:firebase-firestore`.

Favoris

- Les favoris correspondent aux photos likées : requête Firestore `whereArrayContains("likedBy", userId)` sur la collection `photos`, exposée via un Flow temps réel.
Dépendances : `com.google.firebase:firebase-firestore`.

Spécifique TravelPath

Authentification & Gestion du profil

- **Profils enrichis.** Le document `/users/{uid}` contient en plus les champs `fcmToken`, `followersCount`, `followingCount` et un sous-objet `settings`. Les modifications sont gérées depuis `EditProfileScreen`.
- **Statistiques utilisateur.** Les compteurs `followersCount` et `followingCount` sont mis à jour via `FieldValue.increment(±1)` lors de chaque `follow/unfollow`. Le nombre de publications est calculé à la volée par une requête `whereEqualTo("authorId", uid).get()`.
- **Paramètres.** L'`SettingsScreen` expose quatre toggles de notifications gérés localement via `mutableStateOf`, puis sauvegardés en bloc dans Firestore à la confirmation d'une `AlertDialog`.

- **Système d'abonnements (follow/unfollow).** Modélisé par deux sous-collections miroir : `/users/{myId}/following/{targetId}` et `/users/{targetId}/followers/{myId}`. L'état `isFollowing` est initialisé au chargement via `.get().await()`.
Dépendances : `com.google.firebase:firebase-firestore-ktx`, `kotlinx-coroutines-play-services`.
- **Profil public.** `UserDetailDialog` charge le document `/users/{userId}` et le nombre de posts, et embarque un `PostsGallery` filtré par `userIdFilter`.

Création de parcours de visite

- **Parcours en plusieurs étapes.** L'état de création (titre, description, liste de `TravelStep`) est porté par des `mutableStateOf` dans `TravelWowApp` et transmis aux composables `CreatePostContent` et `AddStepScreen`. La confirmation d'une étape ajoute un `TravelStep` via `postSteps + TravelStep(...)`.
- **Géolocalisation (sélection de lieu).** `AddStepScreen` intègre une `GoogleMap` (Compose) permettant de placer une épingle. La distance totale est calculée via la formule de Haversine (`calculateTotalDistance`).
Dépendances : `com.google.maps.android:maps-compose`, `com.google.android.gms:play-services-maps`.
- **Publication Firestore.** Un `DocumentReference` est créé avec `db.collection("travelpath_posts").document()` pour récupérer l'ID avant écriture. Chaque `FirestoreStep` est ensuite écrit dans la sous-collection `/steps/{stepId}`.
- **Brouillons (persistance hybride).** Un `LocalDraft` est d'abord inséré dans Room (stratégie `REPLACE`, `isSynced = false`), garantissant la disponibilité hors-ligne. Une synchronisation Firestore est tentée ensuite ; en cas de succès, le draft est mis à jour avec `isSynced = true`. Les étapes sont sérialisées en JSON via Gson.
Dépendances : `androidx.room:room-runtime`, `androidx.room:room-ktx`, `com.google.code.gson:gson`.

Découverte & Exploration

- **Vue Grille.** Implémentée avec `LazyVerticalGrid` (3 colonnes fixes). Les posts sont chargés depuis Firestore (limite 50 documents) et mélangés aléatoirement (`shuffled()`) pour l'affichage découverte.
- **Vue Carte.** Implémentée avec le SDK `maps-compose`. Lorsqu'un post est sélectionné, ses étapes sont chargées depuis la sous-collection `/steps` et affichées avec des marqueurs bleus ; la caméra effectue une animation `newLatLngBounds` pour englober tous les points du parcours.
- **Filtres avancés.** Le modèle `PostFilter` contient un `Set<String>` de catégories et une plage de distance. Le `FilterBottomSheetContent` utilise un `RangeSlider` et des `FilterChip`. Un point indicateur s'affiche sur l'icône filtre quand `isFilterActive` est vrai.
- **Export PDF.** `PdfExporter` utilise l'API native Android `android.graphics.pdf.PdfDocument`. Les images des étapes sont téléchargées via `ImageLoader` de Coil (`allowHardware(false)`), puis dessinées sur le canvas PDF. La pagination est gérée manuellement.

Organisation sociale & personnelle

- **Système de favoris (cache-then-network).** La base Room est consultée en premier pour un affichage immédiat, puis Firestore met à jour la liste. Les IDs likés sont chargés par chunks de 30 (`whereIn` limité par Firestore). La synchronisation supprime les entrées Room obsolètes.
- **Système de likes.** Le compteur `likesCount` est mis à jour via `FieldValue.increment(±1)` dans le document post. La sous-collection `liked_posts` de l'utilisateur est également mise à jour en parallèle.
- **Partage interne.** `SharePostDialog` fusionne et déduplique les listes d'abonnés et d'abonnements. L'envoi crée un document `FirebaseNotification` (type `SHARE_POST`) consommé par l'écouteur temps-réel du destinataire.
- **Partage externe.** Un `Intent.ACTION_SEND` de type `text/plain` est lancé via `Intent.createChooser`, déléguant à Android le choix de l'application cible.

Infrastructure technique

- **Modèle de données Firestore.** Les entités sont des `data class` Kotlin avec `@DocumentId` pour la résolution automatique de l'ID. Structure des collections : `/users`, `/travelpath_posts` (avec sous-collection `/steps`), `/notifications`.
- **Base de données locale Room.** `TravelWowDatabase` déclare deux entités : `FavoritePost` et `LocalDraft`. Un `TypeConverter` gère la sérialisation des `List<String>` en chaîne CSV. La base est en version 4, avec `fallbackToDestructiveMigration()` activé en développement.
Dépendances : `androidx.room:room-runtime`, `androidx.room:room-ktx`, `androidx.room:room-compiler` (KSP).
- **Navigation et architecture UI.** `NavigationSuiteScaffold` (Material 3 Adaptive) adapte automatiquement la navigation selon la taille d'écran (barre du bas sur mobile, rail latéral sur tablette). Les états imbriqués sont gérés par des flags `Boolean` hoistés dans `TravelWowApp`, avec un `BackHandler`.
Dépendances : `androidx.compose.material3.adaptive:adaptive-navigation-suite`.
- **Firebase App Check.** Initialisé via `DebugAppCheckProviderFactory` dans `TravelWowApplication.onCreate()`. Le provider de debug devra être remplacé par `PlayIntegrityAppCheckProviderFactory` avant la mise en production.
Dépendances : `com.google.firebase:firebase-appcheck-debug`.

Améliorations Potentielles

Général

Cache hors-ligne

- Activer le cache Firestore natif via `FirestoreSettings.Builder().setPersistenceEnabled()` dans `TravelWowApplication` pour conserver automatiquement les dernières données lues.
- Compression et stockage local des médias associés aux contenus sauvegardés en favoris.

Sécurité & Production

- Remplacer le `DebugAppCheckProviderFactory` par `PlayIntegrityAppCheckProviderFactory` avant la mise en production.
- Mettre en place des règles de sécurité Firestore (actuellement ouvertes en mode debug).

Qualité

- Gestion explicite des erreurs réseau avec affichage de Snackbars informatifs (plusieurs blocs `catch` silencieux identifiés).
- Annotations assistées par IA : intégration de l'API Google Cloud Vision ou de l'API Claude pour générer automatiquement des tags et résumés à partir des médias uploadés.

Spécifique TravelShare

Filtrage avancé du feed

- **Filtrage par période / date.** Ajouter un `DateRangePicker` (Material 3). Le champ `timestamp` existe déjà dans `TravelPhoto` ; côté Firestore, utiliser `whereGreaterThanOrEqualTo` + `whereLessThanOrEqualTo` (nécessite un index composite `timestamp` + `isPublic`).
- **Filtrage par auteur.** Cliquer sur le nom d'un auteur dans le feed filtre les résultats via `whereEqualTo("authorId", uid)` — simple ajout dans `FeedViewModel`.
- **Filtrage géographique.** Firestore ne supportant pas nativement les requêtes geo-radius, l'approche standard est `GeoHash` (bibliothèque `firebase/geofire-android`) : encoder latitude/longitude en geohash à la publication, puis requêter par plage de geohash.
- **Flux aléatoire (découverte).** Ajouter un champ `randomScore` : `Double` à chaque document et requêter `whereGreaterThanOrEqualTo("randomScore", Random.nextDouble())` ; ou mélanger côté client via `list.shuffled()`.

Fonctionnalités vocales

- **Recherche vocale.** Intégrer `android.speech.RecognizerIntent` (ou Google Cloud Speech-to-Text) : lancer un `ActivityResultContracts` avec `RecognizerIntent.ACTION_RECOGNIZE_SPEECH` et injecter la chaîne transcrite dans le champ de recherche du `FeedViewModel`.
- **Description vocale sur une photo.** Ajouter un bouton microphone dans `UploadScreen` via `android.media.MediaRecorder`, puis transcrire et insérer le texte dans le champ `description`.

Notifications sur abonnements thématiques

- Ajouter un sous-document `subscriptions` dans `UserProfile` (utilisateurs et tags suivis). Côté backend, une Cloud Function Firebase écoute les créations dans la collection `photos` et crée les `AppNotification` appropriées pour chaque abonné.

Photos similaires

- Utiliser Google Cloud Vision (feature `SIMILAR_IMAGES`) ou un modèle d'embedding `TensorFlow Lite` pour calculer et stocker des vecteurs d'embedding à la publication, puis calculer la similarité cosinus côté client ou via une Cloud Function.

Spécifique TravelPath

Génération intelligente de parcours

- Écran dédié permettant de saisir les critères avant la génération : types d'activités, lieux obligatoires, budget, durée, niveau d'effort et tolérances météorologiques.
- Intégration d'une API de génération (LLM ou service tiers) pour calculer automatiquement des parcours optimisés. Proposer au moins 2 à 3 options avec des profils distincts (« Économique », « Équilibré », « Confort »).
- Prise en compte des horaires d'ouverture et des données météo du jour. Possibilité de régénérer un parcours avec des ajustements.
- Affichage de l'itinéraire complet avec distances, temps de trajet, créneaux horaires suggérés et métriques globales (coût estimé, durée, niveau d'effort).

Interactions sociales à compléter

- Interface utilisateur pour la consultation et la rédaction de commentaires (le modèle `FirestoreComment` et le compteur `commentsCount` existent déjà, mais l'écran est absent).
- Interface de signalement de contenu exploitant le modèle `FirestoreReport` déjà défini.
- Écran de liste des notifications reçues (la collection Firestore `/notifications` est déjà alimentée).

Mode hors-ligne (TravelPath)

- Synchronisation des brouillons non synchronisés (`isSynced = false`) au retour de la connectivité, en exploitant la méthode `getUnsyncedDrafts()` déjà présente dans `DraftDao`.

Qualité & Finitions

- Remplacement de l'icône `ic_return` utilisée comme chevron dans `SettingsNavigationItem` par une icône dédiée.
- Ajout d'une valeur réelle pour `durationMinutes` lors de la création d'un parcours (actuellement fixé à 0).